date : 2026-10-19,21:11

Reference

- video
- Top 10 Proof Techniques NOT Allowed in 6.042
- recitation

OpenCourseWare - Mathematics for Computer Science - 3. Strong induction

Lecture handwritten notes

Date: 18/10/2026

3. Strong induction

3.1 Good proofs Are

- correct
- complete
- clear
- brief
- "elegant"
- well organize
- in order

3.2 proofs techniques you shouldn't use

10. proof by throwing in the kitchen sink.
    copy all the facts so it get point even it can't solveek
9. proof by example
8. proof by vigorous hand-waving
7. proof by cumbersome notation
6. proof by exhaustion
5. proof by omission
4. proof by picture
3. proof by vehement assertion
2. proof by appeal to intuition

SiDU

Date:

1. proof by reference to eminent authority

Fermat's last Thm

$$\forall n > 2 \quad \neg \exists x, y, z \in \eta^{+}$$

$$x^n + y^n = z^n$$

Problem. Find a sequence of move to go from

A	B	C
D	E	F
H	G	

to

A	B	C
D	E	F
G	H	

Legal Move : slide a letter into adjacent blank square

Thm: There is no sequence of legal moves to invent G & H and return all other letter to their original position

invariant: in order to show your system never reach a particular special state. invariant hold at initial state, preserved by every legal move. but that particular don't ~~have that property~~

Date:

Raw move — **Natural order**

e.g

A	B	C
D	G	
E	F	H

→

A	B	C
D		G
F	F	H

1	2	3
4	5	6
7	8	9

order doesn't change

lemma 1: a row move does not change the order of the items.

Proof: in a row move, we move an item from cell i into an adjacent cell $i \pm 1$, Nothing else move Hence the order of items preserved □

colum move

doing those moves to hunt for invariant

E.x

A	B	C
D	F	
H	E	G

→

A	B	C
D	F	G
H	F	

E.x

A	B	C
D		G
E	F	H

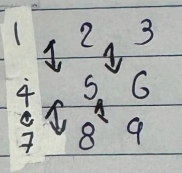
A		C
D	B	G
E	F	H

SiDU

Lemma 2: A column move changes the relative order of precisely 2 pairs of items

Proof: in a column move, we move an item in cell i to a blank spot in cell $i+3$ (or) $i-3$

When an item move 3 positions, it changes order with 2 items $(i \pm 1, i \pm 2)$



Def. A pair of letter, L_1 & L_2 is inversion (a.k.a inverted pair) if L_1 precced L_2 in alphabet but L_1 is after L_2 in puzzle

e.g

A	B	C
F	D	G
E	H	

(D, F), (E, F), (E, G)

Lemm 3 during a move, the # of inversions can only increas by 2, decrease by 2 or stay the same

Proof: row move: no changes (by lemma 1)
column move: 2 pairs change order (lemma 2)

SiDU

A. both pair were in order ⇒ # inversions ↑2

B. both " " inverted ⇒ " " ↓2

C. one pair inverted ⇒ " " stay same

Corollary 1: During a move the parity (even/odd) of # inversions doesn't not change

Pf: adding or subtracting 2 doesn't change parity

invariants

lemma 4: In every state reachable from start stage,

A	B	C
D	E	F
H	G	

, the parity of # inversions is odd

Proof: by induction,

P(n): After any sequence of n moves from the parity of # inversion is odd

Base Case: n=0 # inversions = 1 ⇒ parity odd ✓

the proof By invariant always base case = 0

corollary: a simple consequecen of smth else with short proof

lemma: a tool for bigger thing

inductive step: for n ≥ 0, show P(n) ⇒ P(n+1)

consider any sequence of n+1 moves: $M_1, \ldots, M_{n+1}$

By inductive hypothesis, P(n), we know that the parity after $M_1, \ldots, M_n$ is odd.

By Cor 1, we know the parity of # inversions doesn't change during M_{n+1} ⇒ the parity after $M_1, \ldots, M_n, M_{n+1}$ is odd = P(n+1) □

Pf of thm The parity of # inversions in desired state is even (0). By lemma 4, the desired state cannot be reached from $\begin{pmatrix} A & B & C \\ D & E & F \\ H & G & \end{pmatrix}$ □

invariants: use to prove your system never reach a particular state

By finding invariant hold by initial state, preserved by every legal move and that particular state never have

strong induction Axiom

Let $P(n)$ be any predicate, If $P(0)$ is true & $\forall n\ (P(0) \wedge P(1) \wedge \dots \wedge P(n)) \Rightarrow P(n+1)$ is true then $\forall n\ P(n)$ is true

- we are gonna assume $(P(0) \wedge P(1) \wedge P(2) \wedge \dots \wedge P(n))$ is true
- stronger and easier to proof

Thm All strogies for the n-block game produce the same score, $S(n)$

Ex : $S(8) = 28$

Proof: By strong induction

I.H

Base Case : $n=1$ $\quad S(1) = 0$ ✓

Inductive step : Assume $P(1), P(2), \dots, P(n)$ to prove $P(n+1)$

look at $n+1$ blocks $\quad n+1 \quad 1 \le k \le n$

$k \quad n+1-k$

$S(n+1)$

Score $= k(n+1-k) + P(k) + P(n+1-k)$

since it is still depend on k only n so we will use stronger induction hypothesis

Thm: $S(n) = \frac{n(n-1)}{2}$ $\quad$ checking $S(8) = \frac{8(7)}{2} = 28$

$S(2) = \frac{2(1)}{2} = 1$

Base case $n=1$ $\quad S(1) = 0$ ✓ $\quad S(3) = 3$

inductive case

Assume $P(1), P(2), \dots, P(n)$ to prove $P(n+1)$

$$\text{Score} = k(n+1-k) + P(k) + P(n+1-k)$$

$$= k(n+1-k) + \frac{k(k-1)}{2} + \frac{(n+1-k)(n-k)}{2}$$

$$= \frac{2kn + 2k - 2k + k^2 - k + n(n+1) - kn - k + k^2}{2}$$

$$= S(n+1)$$

SiDU

Book reading

3.3 Invariants

One of the most important uses of induction in computer science involves proving that a program or process preserves one or more desirable properties as it proceeds. A property that is preserved through a series of operations or steps is known as an invariant.

induction & invariants

We typically use induction to prove that a proposition is an invariant.
In particular, we show that the proposition is true at the beginning (this is the base case) and that if it is true after t steps have been taken, it will also be true after step $t+1$(this is the inductive step).

proving proposition is indeed an invariant, namely, that it will always hold.

3.3.1 A Simple Example: The Diagonally-Moving Robot

> Invariants are useful in systems that have a start state (or starting configuration) and a well-defined series of steps during which the system can change state.

---

problem :

Suppose that you have a robot that can walk across diagonals on an infinite 2-dimensional grid. The robot starts at position $(0,0)$ and at each step it moves up or down by 1 unit vertically and left or right by 1 unit horizontally. To be clear, the robot must move by exactly 1 unit in each dimension during each step, since it can only traverse diagonals.

In this example, the *state* of the robot at any time can be specified by a coordinate pair (x,y) that denotes the robot's position. The *start state* is $(0,0)$ since it is given that the robot starts at that position. After the first step, the robot could be in states $(1,1)$, $(1,-1)$, $(-1,1)$, or $(-1,-1)$. After two steps, there are 9 possible states for the robot, including $(0,0)$.

Can the robot ever reach position $(1,0)$?

> you need to get example so you will get starting points or lemma

After playing around with the robot for a bit, it will become apparent that the robot will never be able to reach position $(1, 0)$. This is because the robot can only reach positions (x, y) for which $x + y$ is even. This crucial observation quickly leads to the formulation of a predicate

$$P(t) :: \text{if the robot is in state } (x, y) \text{ after } t \text{ steps, then } x + y \text{ is even}$$

which we can prove to be an invariant by induction.

Theorem 3.3.1. *The sum of robot's coordinates is always even.*

Proof. We will prove that P is an invariant by induction.

$P(0)$ is true since the robot starts at $(0, 0)$ and $0 + 0$ is even.

Assume that $P(t)$ is true for the inductive step. Let (x, y) be the position of the robot after t steps. Since $P(t)$ is assumed to be true, we know that $x + y$ is even. There are four cases to consider for step $t + 1$, depending on which direction the robot moves.

Case 1 The robot moves to $(x + 1, y + 1)$. Then the sum of the coordinates is $x + y + 2$, which is even, and so $P(t + 1)$ is true.

Case 2 The robot moves to $(x+1, y-1)$. The the sum of the coordinates is $x+y$, which is even, and so $P(t + 1)$ is true.

Case 3 The robot moves to $(x-1, y+1)$. The the sum of the coordinates is $x+y$, as with Case 2, and so $P(t+1)$ is true.

Case 4 The robot moves to $(x-1, y-1)$. The the sum of the coordinates is $x+y-2$, which is even, and so $P(t+1)$ is true.

In every case, $P(t+1)$ is true and so we have proved $P(t)$ IMPLIES $P(t+1)$ and so, by induction, we know that $P(t)$ is true for all $t \geq 0$. ■

Corollary 3.3.2. *The robot can never reach position* $(1, 0)$.

Proof. By Theorem 3.3.1, we know the robot can only reach positions with coordinates that sum to an even number, and thus it cannot reach position $(1, 0)$. ■

Since this was the first time we proved that a predicate was an invariant, we were careful to go through all four cases in gory detail. As you become more experienced with such proofs, you will likely become more brief as well. Indeed, if we were going through the proof again at a later point in the text, we might simply note that the sum of the coordinates after step $t+1$ can be only $x+y$, $x+y+2$ or $x+y-2$ and therefore that the sum is even.

3.3.2 The Invariant Method

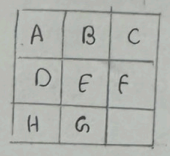
In summary, if you would like to prove that some property NICE holds for every step of a process, then it is often helpful to use the following method:

- Define $P(t)$ to be the predicate that NICE holds immediately after step t.
- Show that $P(0)$ is true, namely that NICE holds for the start state.
- Show that

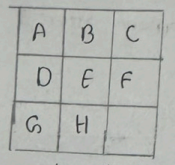
$$\forall t \in \mathbb{N}.\ P(t) \text{ IMPLIES } P(t+1),$$

  namely, that for any $t \geq 0$, if NICE holds immediately after step t, it must also hold after the following step.

3.3.4 The 8-Puzzle



initial stage



desired stage

Thm: No sequence of legal move transform the configuration in intial stage to desired stage

> Play around to get lemma(s)

Lemma 1: A row move doesn't change the order of tiles

Proof: A row move moves a tile from cell i to cell $i \pm 1$, This doesn't chage its order with respect to any other tile. Since no other tile move, there is no change in the order of any of the other pairs of tiles

D E
H . .

→

D . E
H

D E H still the same

Lemma 2: A column move changes the relative order of exactly two pairs of ties

Proof: Sliding a tie down move it cell i to $i \pm 3$, when on item move 3 positions, it change the order with two item ($i \pm 1$ and $i \pm 2$)

A B
D E C

B ⇒ D, E, C
D ⇒ E, C
E ⇒ C

→

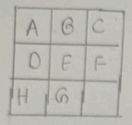
A B C
D E

B → D, E C
C → D, E
D - E

Change
D, C → C, D
E, C → C, E

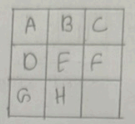
Def: A pair of letters, L_1 & L_2 is an inversion if L_1 preceeds L_2 in the Alphabet, but L_1 appear after L_2 in the puzzle order

(we will only → check)



initial stage

1 inversion (H, G)



desired stage

no inversion

Lemma 3: During a move, # of inversion can only incearse by 2 or decrease by 2 or stay the same

Proof: raw move : no changes (lemma 1)
column move: 2 changes of order (Lemma 2)

A. if both pairs were in order → # inversion ↑ 2
B. if both " inverted → # " ↓ 2
C. if one pair inverted → # " same

Corollay 1: During a move, the parity of # inversion doesn't change

Proof: adding or subtracting 2 does not change parity

Lemma 4: in every state reachable from initial stage, # of inversion is odd

invariant

Proof: by induction

P(n): After any sequence of n moves from initial stage, the parity of # inversion is odd

Base case : $n = 0 \Rightarrow$ # of inversion = 1 $\Rightarrow$ parity odd

the proof by invaiant always start at base case = 0

inductive steps : for $n \geq 0$, show $P(n) \Rightarrow P(n+1)$

assume $P(n)$ = True

$n+1$ moves $\Rightarrow m_1, \cdots, m_n, m_{n+1}$

By inductive hypothesis, $P(n)$, we know $m_1, \cdots, m_n$ is odd

By corollay 1, the parity of # of inversion stay the some during any move, m_{n+1}

therfore, $m_1, \cdots, m_{n+1}$ is odd

$P(n) \Rightarrow P(n+1)$

By the principle of induction, $P(n)$ is true for all $n \geq 0$.

Pf for thm : the parity of # of inversion in desired state is even (0). By lemma 4, the desired state can't be reach from initial stge. which the parity of # of inversion is odd.

3.4 Strong Induction

> a variation of ordinary induction that is useful when the predicate $P(n+1)$ naturally depends on $P(a)$ for values of $a < n$.

3.4.1 A Rule for Strong Induction

Principle of Strong Induction. Let $P(n)$ be a predicate. If

- $P(0)$ is true, and
- for all $n \in \mathbb{N}$, $P(0), P(1), \ldots, P(n)$ *together* imply $P(n+1)$,

then $P(n)$ is true for all $n \in \mathbb{N}$.

rule for strong induction

Rule. ***Strong Induction Rule***

$$\frac{P(0), \quad \forall n \in \mathbb{N}.\ \big(P(0) \text{ AND } P(1) \text{ AND } \ldots \text{ AND } P(m)\big) \text{ IMPLIES } P(n+1)]}{\forall m \in \mathbb{N}.\ P(m)}$$

The template for strong induction proofs is identical to the template given in Section 3.2.3 for ordinary induction except for two things:

- you should state that your proof is by strong induction, and
- you can assume that $P(0)$, $P(1)$, ..., $P(n)$ are all true instead of only $P(n)$ during the inductive step.

3.4.2 Some Examples

Thm: every integer ≥ 2 is a product of prime

Pf: by strong induction

$P(n)$: n is a product of primes

Base case:

$P(2)$ = True

inductive step:

$n \geq 2$, show $(P(2) \wedge \dots \wedge P(n)) \Rightarrow P(n+1)$

Assume $P(2) \wedge \dots \wedge P(n)$ is true for the purpose of induction

case A:

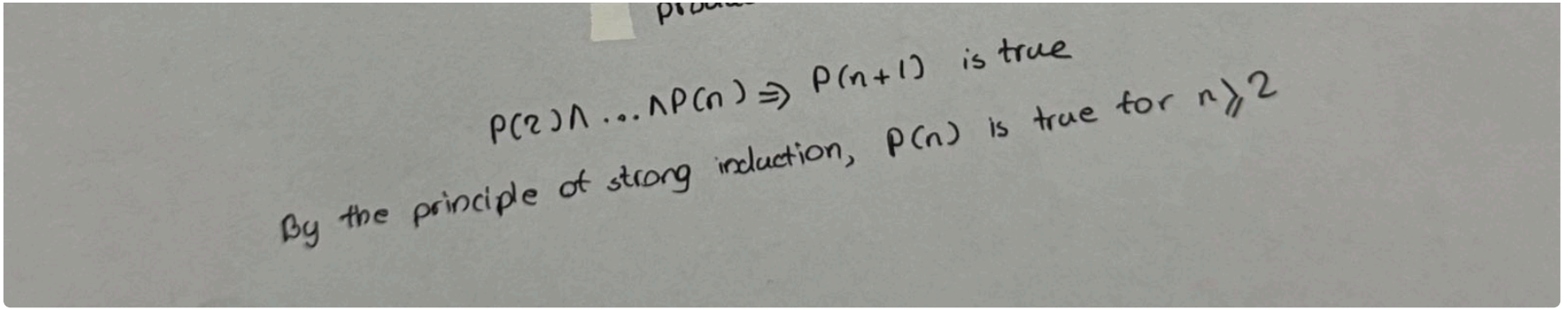
If $(n+1)$ itself is prime, $1 \times$ prime is a length one product of prime

so $P(n+1)$ is True

case B:

If $(n+1)$ is not a prime, $n+1 = km$ where k, m are integers that $2 \leq m, k < k$

By induction hypothesis, $P(2) \wedge \dots \wedge P(n)$, m and k are products of primes therefore k is a product of prime



3.4.3 Strong Induction versus Induction

any proof using strong induction can be reformatted into a proof using ordinary induction.

Which method should you use? Whichever you find easier.

3.5 Structural Induction

> the idea of induction is far more general—it can be applied to a much richer class of sets in particular with sets or data types that are defined recursively.

3.5.1 Recursive Data Types

Recursive data types play a central role in programming. They are specified by *recursive definitions* that say how to build something from its parts. *Recursive* definitions have two parts:

- **Base case(s)** that don't depend on anything else.
- **Constructor case(s)** that depend on previous cases.

Let's see how this works in a couple of examples: Strings of brackets and expression evaluation.

Definition 3.5.1. The set `brkts` of strings of brackets can be defined recursively as follows:

- **Base case:** The *empty string*, λ, is in `brkts`.
- **Constructor case:** If $s \in$ `brkts`, then s**]** and s**[** are in `brkts`.

Definition 3.5.2. Recursively define the set, RecMatch, of strings as follows:

- **Base case:** $\lambda \in \text{RecMatch}$.
- **Constructor case:** If $s, t \in \text{RecMatch}$, then

$$[\,s\,]\,t \in \text{RecMatch}.$$

Here we're writing $[\,s\,]\,t$ to indicate the string that starts with a left bracket, followed by the sequence of brackets (if any) in the string s, followed by a right bracket, and ending with the sequence of brackets in the string t.

Using this definition, we can see that $\lambda \in \text{RecMatch}$ by the Base case, so

$$[\,\lambda\,]\,\lambda = [\,] \in \text{RecMatch}$$

by the Constructor case. So now,

$$\begin{aligned} [\,\lambda\,][\,] = [\,][\,] &\in \text{RecMatch} && (\text{letting } s = \lambda, t = [\,]) \\ [\,[\,]\,]\,\lambda = [\,[\,]\,] &\in \text{RecMatch} && (\text{letting } s = [\,], t = \lambda) \\ [\,[\,]\,][\,] &\in \text{RecMatch} && (\text{letting } s = [\,], t = [\,]) \end{aligned}$$

now we can use **RecMatch**, an constructor to create and describe infinite set `brkts`

3.5.2 Structural Induction on Recursive Data Types

Structural induction is a method for proving that some property, P, holds for all the elements of a recursively-defined data type. The proof consists of two steps:

- Prove P for the base cases of the definition.
- Prove P for the constructor cases of the definition, assuming that it is true for the component data items.

The theorem states that **every string s in `RecMatch` has an equal number of left brackets `[` and right brackets `]`.**

1. The Setup and Predicate

First, we define our induction predicate $P(s)$:

- $P(s)$: "s has an equal number of left and right brackets" (or mathematically, $\#s] = \#[s$).

To prove this for all $s \in \text{RecMatch}$ using structural induction, we need to check two things:

1. **The Base Case:** Show that the property holds for the fundamental starting element.
2. **The Constructor Case:** Show that if the property holds for smaller component strings (s and t), it continues to hold when we build a larger string using the recursive rule.

2. Step 1: The Base Case

- **What `RecMatch` defines as the base case:** The empty string ϵ is in `RecMatch`.
- **The Proof:** For the empty string ϵ:

$$\#[\epsilon = 0 \quad \text{and} \quad \#]\epsilon = 0$$

Since $0 = 0$, the base case $P(\epsilon)$ is true.

3. Step 2: The Constructor Case

- **What `RecMatch` defines as the constructor rule:** If s and t are already valid strings in `RecMatch`, then sticking them together into the new form $[s]t$ is also in `RecMatch`.
- **The Induction Hypothesis (IH):** We *assume* that our predicate is already true for the smaller component parts s and t. That means:
  - s has an equal number of left and right brackets: $\#s] = \#[s$
  - t has an equal number of left and right brackets: $\#t] = \#[t$
- **Our Goal:** Prove that the new combined string $[s]t$ also has an equal number of left and right brackets (i.e., $P([s]t)$ is true).
- **Counting the Brackets in $[s]t$:**
  1. **Left brackets (`[`):** The new string has the extra leading `[`, plus all the left brackets inside s and t:

$$\#[_{[s]t} = 1 + \#[_s + \#[_t$$

  2. Right brackets (`]`):** The new string has the extra trailing `]`, plus all the right brackets inside s and t:

$$\#[_{[s]t} = 1 + \#]_s + \#]_s$$

- **Using the Hypothesis:** Because our induction hypothesis tells us that $\#s] = \#[s$ and $\#t] = \#[t$, we can substitute them into the right-bracket equation:

Both sides equal each other! This proves that the newly constructed string $[s]t$ also has a balanced number of brackets.

3.5.3 Functions on Recursively-defined Data Types

Recursively-Defined Functions

To define a recursive function „on a recursive data type, you only need to specify two things:

1. **The Base Cases:** Define the value of „directly for the simplest, atomic forms of the data type.
2. **The Constructor Cases:** Define the value of „for complex structures in terms of the values of „operating on their smaller component parts.

an example

$$\text{eval } .\text{Aexp} \times \mathbb{Z} \to \mathbb{Z}$$

`Aexp` : **Arithmetic Expressions**.

- It represents the set of all valid mathematical expression structures you can build using numbers, the variable x, and basic operations (like addition, multiplication, and negation).
- e.g. $(x + 3)x$

Z : a set of integer

defining a recursive function

Definition 3.5.5. The *evaluation function*, eval : Aexp $\times \mathbb{Z} \to \mathbb{Z}$, is defined recursively on expressions, $e \in$ Aexp, as follows. Let n be any integer.

- **Base cases:**

  1. Case[e is x]

     $$\text{eval}(x, n) ::= n.$$

     (The value of the variable, x, is given to be n.)

  2. Case[e is k]

     $$\text{eval}(\mathtt{k}, n) ::= k.$$

     (The value of the numeral $\mathtt{k}$ is the integer k, no matter what value x has.)

- **Constructor cases:**

  3. Case[e is $(e_1 + e_2)$]

     $$\text{eval}((e_1 + e_2), n) ::= \text{eval}(e_1, n) + \text{eval}(e_2, n).$$

  4. Case[e is $(e_1 * e_2)$]

     $$\text{eval}((e_1 * e_2), n) ::= \text{eval}(e_1, n) \cdot \text{eval}(e_2, n).$$

  5. Case[e is $(-e_1)$]

3. Case[e is $-(e_1)$]

$$\text{eval}(-(e_1), n) ::= -\text{eval}(e_1, n).$$

For example, here's how the recursive definition of eval would arrive at the value of $3 + x^2$ when x is 2:

$$\begin{aligned}
\text{eval}((3 + (x * x)), 2) &= \text{eval}(3, 2) + \text{eval}((x * x), 2) && \text{(by Def 3.5.5.3)} \\
&= 3 + \text{eval}((x * x), 2) && \text{(by Def 3.5.5.2)} \\
&= 3 + (\text{eval}(x, 2) \cdot \text{eval}(x, 2)) && \text{(by Def 3.5.5.4)} \\
&= 3 + (2 \cdot 2) && \text{(by Def 3.5.5.1)} \\
&= 3 + 4 = 7.
\end{aligned}$$

Ambiguity

ambiguous: When a recursive definition of a data type allows the same element to be constructed in more than one way.

- A function defined recursively from an ambiguous definition of a data type will not be well-defined unless the values specified for the different ways of constructing the element agree.

unambiguous definitions—so that every element has one unique, undeniable way of being constructed, ensuring that any function you write will always return a consistent, predictable answer.

- If a data type definition is **ambiguous**, any recursively-defined function you write for it **might not be well-defined**.

- This means if a function tries to process a string, it could arrive at **conflicting answers** depending entirely on *which* construction path was used to build that string.

3.5.4 Recursive Functions on N —Structural Induction versus Ordinary Induction

The nonnegative integers can be understood as a recursive data type.

Definition 3.5.8. The set, $\mathbb{N}$, is a data type defined recursivly as:

- **Base Case:** $0 \in \mathbb{N}$.
- **Constructor Case:** If $n \in \mathbb{N}$, then the *successor*, $n + 1$, of n is in $\mathbb{N}$.

> ordinary induction is a special case of structural induction on the recursive Definition 3.5.8.

Conversely, most proofs based on structural induction that you will encounter in computer science can also be reformatted into proofs that use only ordinary induction.

The decision as to which technique to use is up to you, but it will often be the case that structural induction provides the easiest approach when you are dealing with recursive data structures or functions

Some famous number order as recursive data types

The Factorial Function

The factorial function is often written "$n!$." You will be seeing it a lot in Parts III and IV of this text. For now, we'll use the notation $\text{fac}(n)$ and define it recursively as follows:

- **Base Case:** $\text{fac}(0) ::= 1$.
- **Constructor Case:** $\text{fac}(n+1) ::= (n+1) \cdot \text{fac}(n)$ for $n \geq 0$.

The Fibonacci numbers.

Fibonacci numbers arose out of an effort 800 years ago to model population growth. We will study them at some length in Part III. The nth Fibonacci number, $\text{fib}(n)$, can be defined recursively by:

- **Base Cases:** $\text{fib}(0) ::= 0$ and $\text{fib}(1) ::= 1$
- **Constructor Case:** $\text{fib}(n) ::= \text{fib}(n-1) + \text{fib}(n-2)$ for $n \geq 2$.

Here the recursive step starts at $n = 2$ with base cases for $n = 0$ and $n = 1$. This is needed since the recursion relies on two previous values.

What is $\text{fib}(4)$? Well, $\text{fib}(2) = \text{fib}(1) + \text{fib}(0) = 1$, $\text{fib}(3) = \text{fib}(2) + \text{fib}(1) = 2$, so $\text{fib}(4) = 3$. The sequence starts out $0, 1, 1, 2, 3, 5, 8, 13, 21, \ldots$.

Sum-notation

Let "$S(n)$" abbreviate the expression "$\sum_{i=1}^{n} f(i)$." We can recursively define $S(n)$ with the rules

- **Base Case:** $S(0) ::= 0$.
- **Constructor Case:** $S(n+1) ::= f(n+1) + S(n)$ for $n \geq 0$.

Wrong definition

Ill-formed Function Definitions

There are some blunders to watch out for when defining functions recursively. Below are some function specifications that resemble good definitions of functions on the nonnegative integers, but they aren't.

Definition 3.5.9.

$$f_1(n) ::= 2 + f_1(n-1). \tag{3.7}$$

This "definition" has no base case. If some function, f_1, satisfied (3.7), so would a function obtained by adding a constant to the value of f_1. So equation (3.7) does not uniquely define an f_1.

Definition 3.5.10.

$$f_2(n) ::= \begin{cases} 0, & \text{if } n = 0, \\ f_2(n+1) & \text{otherwise.} \end{cases} \tag{3.8}$$

This "definition" has a base case, but still doesn't uniquely determine f_2. Any function that is 0 at 0 and constant everywhere else would satisfy the specification, so (3.8) also does not uniquely define anything.

In a typical programming language, evaluation of $f_2(1)$ would begin with a recursive call of $f_2(2)$, which would lead to a recursive call of $f_2(3)$, ... with recursive calls continuing without end. This "operational" approach interprets (3.8) as defining a *partial* function, f_2, that is undefined everywhere but 0.

Definition 3.5.11.

$$f_3(n) ::= \begin{cases} 0, & \text{if } n \text{ is divisible by 2,} \\ 1, & \text{if } n \text{ is divisible by 3,} \\ 2, & \text{otherwise.} \end{cases} \tag{3.9}$$

This "definition" is inconsistent: it requires $f_3(6) = 0$ and $f_3(6) = 1$, so (3.9) doesn't define anything.

A Mysterious Function

Mathematicians have been wondering about the following function specification for many years:

$$f_4(n) ::= \begin{cases} 1, & \text{if } n \leq 1, \\ f_4(n/2) & \text{if } n > 1 \text{ is even,} \\ f_4(3n+1) & \text{if } n > 1 \text{ is odd.} \end{cases} \tag{3.10}$$

Notes for Recitation 3

1. State Machine

A state machine is an abstract model of a step-by-step process. The model consists of a collection of states and transitions between those states.

More formally, we can define a state machine as

- a set of states: Q
- a designated start state: $_0 \in Q$
- a set of allowed transitions between states: $\delta \subseteq Q \times Q$.

example

Take, for example, a bounded counter that counts from 0 to 99 and overflows at 100.

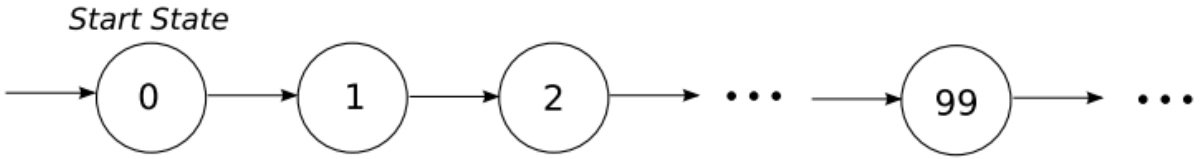


Figure 1: State machine for the unbounded counter

3 Problem: The Temple of Forever

Each monk entering the Temple of Forever is given a bowl with 15 red beads and 12 green beads. Each time the Gong of Time rings, a monk must do one of two things:

1. *Exchange*: If he has at least 3 red beads in his bowl, then he may exchange 3 red beads for 2 green beads.

2. *Swap*: He may replace each green bead in his bowl with a red bead and replace each red bead in his bowl with a green bead. That is, if he starts with i red beads and j green beads, then after he performs this operation, he will have j red beads and i green beads.

A monk may leave the Temple of Forever only when he has exactly 5 red beads and 5 green beads in his bowl.

Let's look at how we can represent this problem as a state machine.

A monk may leave the Temple of Forever only when he has exactly 5 red beads and 5 green beads in his bowl.

Let's look at how we can represent this problem as a state machine.

- What do the states of the machine look like?

  Solution. We can use variables such as r to represent the number of red beads and g to represent the number of green beads. Then the states can be represented as pairs (r, g) for $r \geq 0, g \geq 0$. ■

- Use the notation you developed above to represent the allowable transitions in the state machine.

  Solution. There are two possible transitions:

  1. trade 3 red for 2 green (*exchange*): $(r, g) \to (r - 3, g + 2), r \geq 3$
  2. swap red and green (*swap*): $(r, g) \to (g, r)$

  ■

- Expand the state machine diagram to the first three or four levels. Label the transitions according to the operation type (E for *exchange* or S for *swap*).

  Solution. You can see the diagram in Figure 3.

  ■

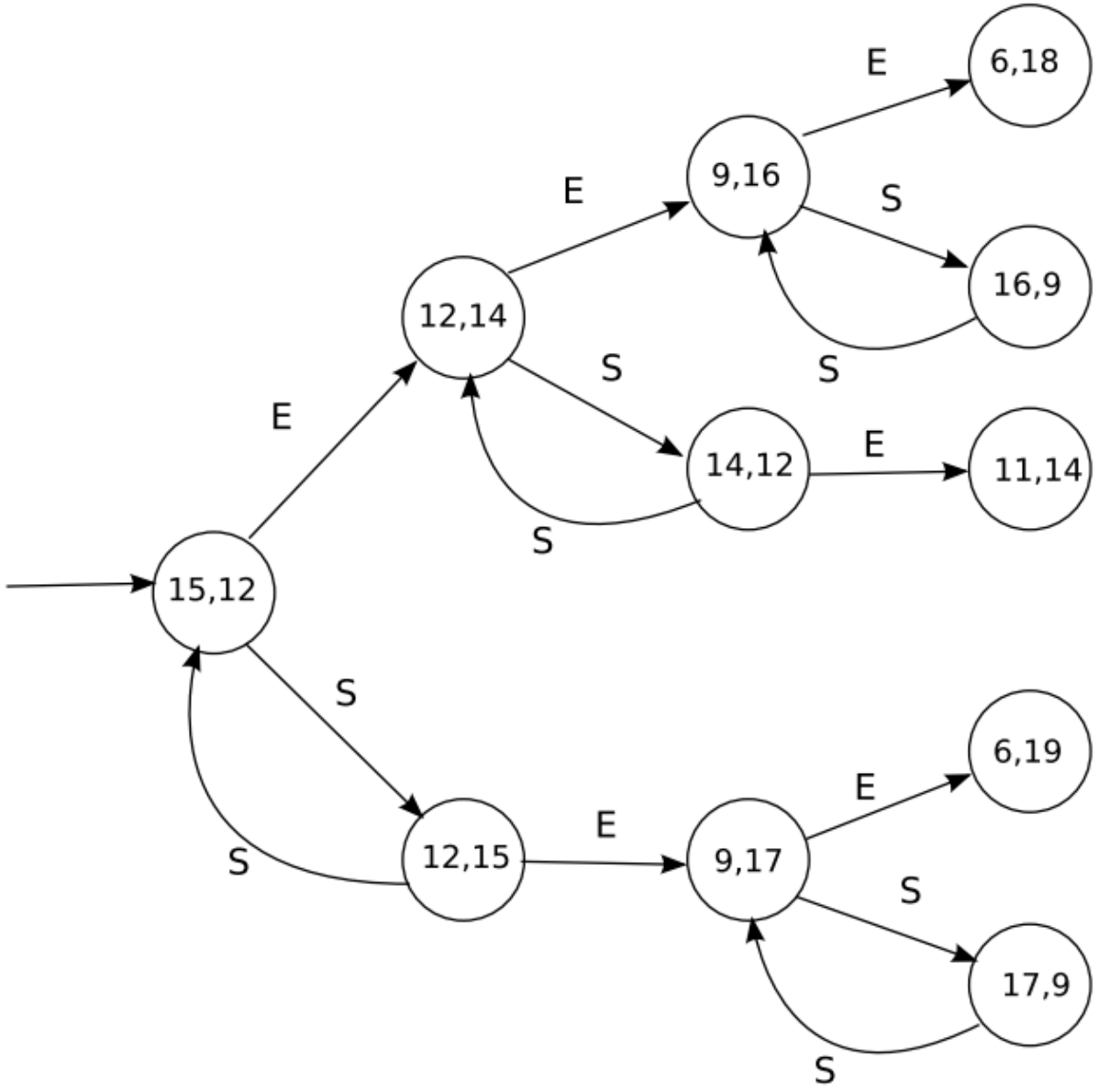

15,12
E
12,14
E
9,16
E
6,18
S
16,9
S
S
14,12
E
11,14
S
S
12,15
E
9,17
E
6,19
S
17,9
S

Figure 2: State machine diagram for first few levels

3.1 Invariants and State Machines

The Temple of Forever machine models every possible way, *executions of the machine*, of exchanging beads at each time step according to the rules.

if the state we are interested in appears in some execution of the machine. If so, then we can say that that state is *reachable*.

Definition 1. *A state is called* reachable *if there is a path to it starting from the start state, that is, if it appears in some execution.*

A useful technique for analyzing reachability is the identification of *invariants* of the machine. An invariant will hold for all reachable states of the machine. More formally,

Definition 2. *An* invariant *for a state machine is a predicate P on state machines such that $P(q_0)$ holds (where q_0 is the start state of the machine), and whenever $P(q)$ is true of a state q, and $q \to r$ for some state r, then $P(r)$ holds.*

Since, by definition, an invariant holds for the start state and for all transitions of the state machine, then we know (by induction) that the invariant must hold for all reachable states. Therefore, if we know some property to be an invariant, and we know that a certain state violates that property, then we can say that this state is unreachable.

proving theorem 1

Theorem 1. No one ever leaves the Temple of Forever.

Thm: No one ever leaves the Temple of Forever

Poof: By induction

Let P(n) is # of red beads − # of blue beads after (n) rings:

$P(n) = 5k+2$ (or) $5k+3$ for some integer k

Base Case

$P(0) = 15 - 12 = 3$

we will use $k=0$, $5.0+3=3$

Inductive steps

$n \geq 0$, show $P(n) \rightarrow P(n+1)$

assume $P(n)$ is true for the purpose of induction

r = # of red beads } of $P(n)$

g = # of green beads }

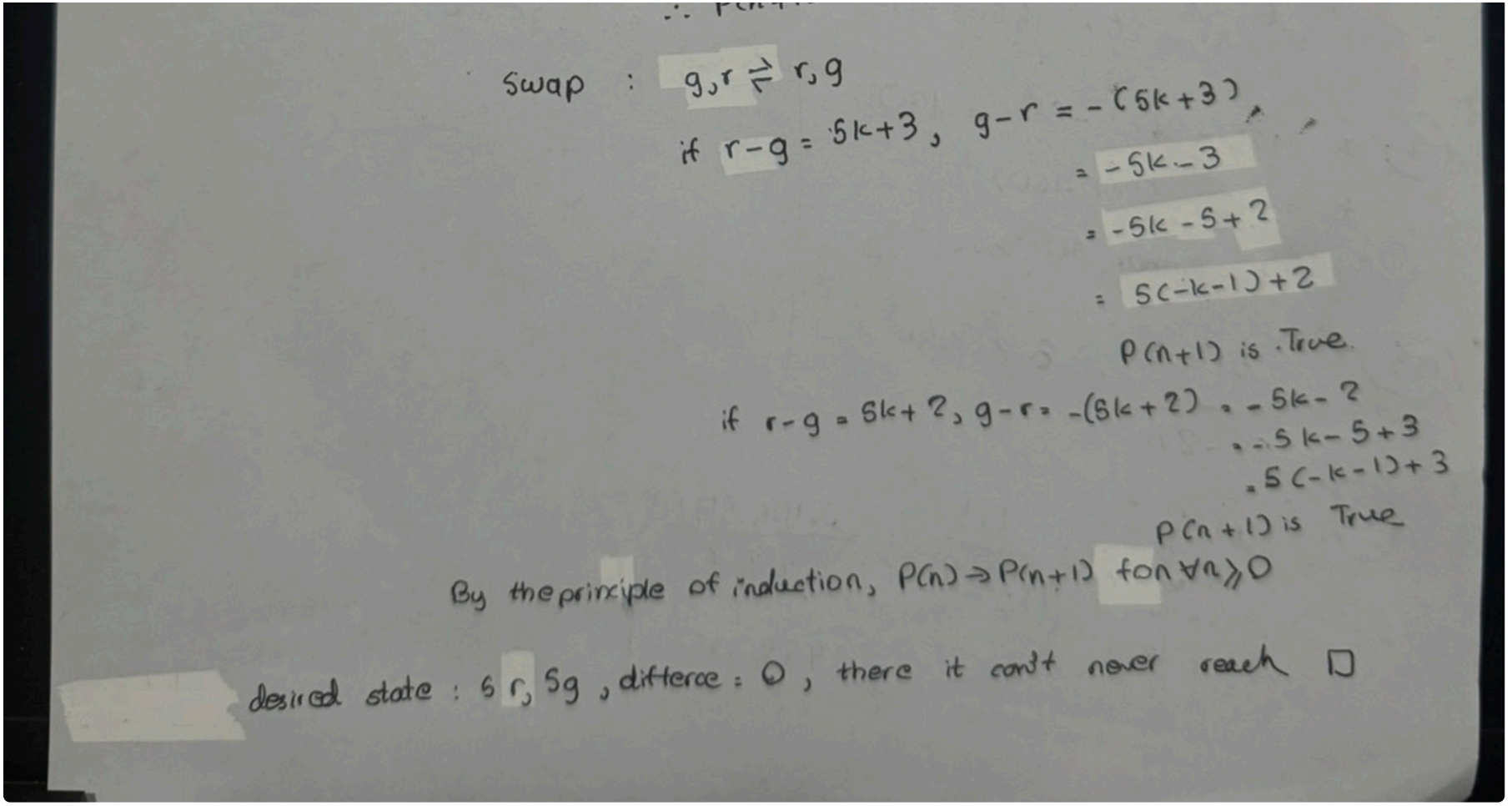
By inductive hypothesis, we know $P(n)$ is true, to make $P(n+1)$ we will make one extra move from $P(n)$

Exchange : $(r-3)-(g+2) = (r-g)-5$

since $(r-g)$ which is equal to $5k+2$ (or) $5k+3$ by inductive hypothesis,

$(r-g)-5$ will always be equal to $5k+2$ (or) $5k+3$

$P(n+1)$ is true



proving theorem 2

Thm: There is a finite number of reachable states in the temple of fover machine

Lemma 1: For all reachable state, the total number of red beads and green beads; r+g is at most 27

Proof: by induction

$P(n)$: $r+g \leq 27$ after n rings

Base case: $P(0) = 15+27 = 27$ ✓

inductive step: $n \geq 0$, show $P(n) \Rightarrow P(n+1)$

Assume $P(n)$ is true for the purpose of induction,

since $P(n) \leq 27$, $P(n+1)$ is next step after $P(n)$

denoate r = # of red beans after $P(n)$

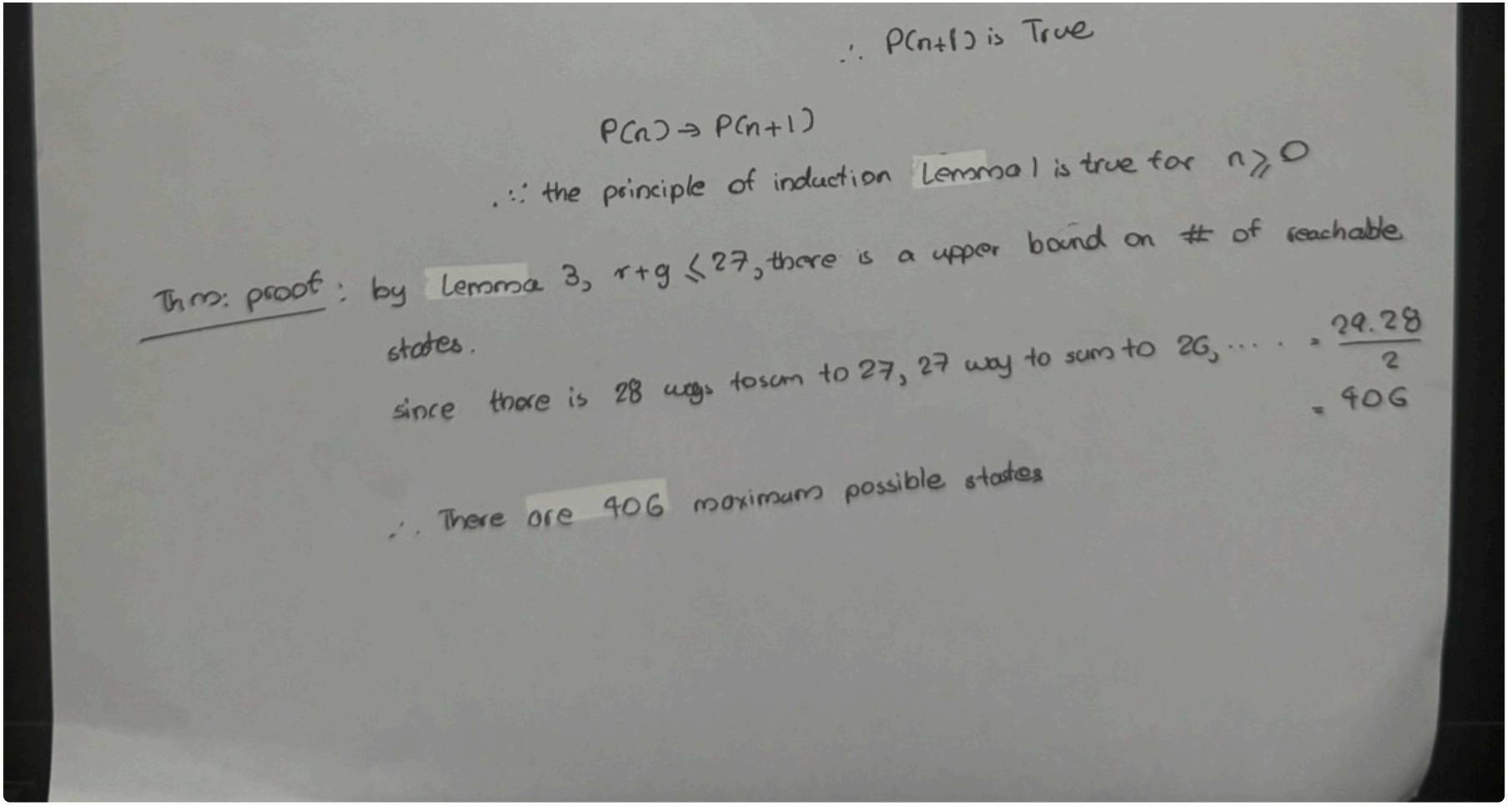
g = # of green beans after $P(n)$

there are two possible steps at $P(n+1)$

Exchange: $(r-3)+(g+2) = (r+g)-1$

$\therefore$ $P(n+1)$ is True

Swapp: $g+r = r+g$



proving theorem 3

Thm: It is not possible to visit 108 unique states in the Temple of Forever machine

Proof: By contradiction.

Assume a monk can visit all 108 unique states by legal moves, swap & exchange

if monk exchange, the sum of r+g is decreased by 1

$$(r-3)+(g+2)=(r+g)-1$$

if monk swap, there is no effect on the sum.

Beause they can only exchange $r \geq 3$ and start with $r+g=27$, each change cost 1 beads. So they can only pertorm at most 25 exchanges

swap: if you swap, $(r,g) \to (g,r)$. if you swap again, it will move back to original $(g,r) \to (r,g)$. meaning you can have at most 1 new unique state before go back to original.

Therefore, an upper bond is $25+25+1+1=52$ (a swap at beginning & at the end)

This contrdict the assumption that the monk visits 108 unique states

Theretor no monk naver leaves the temple of forever.